

Vereins-OS / M75-Manager — Technische Gesamtdokumentation

Erstellt: 29. Juni 2026 · **Autor:** Cascade (KI-Assistenz) **Basis:** Code-Analyse des Repos `Netjogger58/Vereins-OS` + eingereichter Vision-Entwurf (29.06.2026, 14:24 CEST). **Zweck:** Vollständige, **ehrliche** technische Spezifikation — was real implementiert ist, was nur als Datenmodell existiert und was Vision/Roadmap ist.

0. Status-Legende (gilt für das ganze Dokument)

Symbol	Bedeutung
✓	Fertig & nutzbar — im Code vorhanden und einsetzbar
■	Funktioniert, ausbaufähig — UI + Logik da, Luft nach oben
●	Platzhalter-Stub — Seite existiert, aber generisches CRUD ohne Fachlogik
⚙	Nur Datenmodell — DB-Tabelle existiert, keine UI
📅	Konzept / Vision / Roadmap — kein Code , nur geplant

Wichtig: Der eingereichte Entwurf hat mehrere geplante Bausteine wie fertige Module beschrieben. Dieses Dokument korrigiert das (siehe §0.1) und kennzeichnet jeden Baustein mit obiger Legende.

0.1 Korrekturen gegenüber dem eingereichten Entwurf (verifiziert am Code)

Aussage im Entwurf	Status laut Code	Korrektur
„Öffentliche Website = WordPress auf Hetzner"	✗ falsch	Website ist statisches HTML/CSS/Vanilla-JS auf GitHub Pages (Domain <code>mersch75.lu</code> via <code>CNAME</code>). Kein WordPress im Code.
Hermes & Odysseus als „Basis der Architektur"	Konzept	Beide sind geplant, kein Code . Aktuell läuft der FLH-Import als Server-Modul (<code>flhImport.ts</code>), nicht als Playwright-Agent.
Kassensysteme Getränke & Eintritt (Strichliste, Ticketverkauf)	Idee	Keine Tabellen (<code>drink/ticket/entrance/kiosk</code>) im Schema. Nur <code>qr_checkins</code> + <code>member_cards</code> für Check-in vorhanden.
Payroll/Gehälter, Subventionen/Fördermittel, Guichet-DMS	Idee	Keine payroll/salary/subvention/grant -Tabellen. Nur generische <code>transactions</code> .
Sponsoring mit ROI-/Leistungstracking	🟡 Stub	<code>sponsors</code> -Tabelle + <code>Sponsors.tsx</code> existieren, aber nur als generischer Stub (Name/Beschreibung). ROI ist Ausbau-Idee.
Bénévole-Stundenerfassung & Statistik		<code>duties</code> , <code>duty_rosters</code> , <code>duty_swaps</code> als Tabellen vorhanden; Stundenstatistik/Aufwandsentschädigung nicht modelliert .
Headless-API speist Website automatisch	Konzept	<code>website_pages</code> -Tabelle + <code>Website.tsx</code> (iframe-Hub) vorhanden, aber keine öffentliche Read-API implementiert.
QNAP-Offsite-Backup (192.168.178.5)	Konzept	Kein Backup-Code im Repo; sinnvolle Roadmap, aber noch nicht gebaut.
Ollama, Matrix, Element, n8n, Uptime Kuma, Mautrix, Nextcloud	Konzept	Alle geplant, kein Code . Siehe §11.

1. Sinn, Zweck & Host-Infrastruktur

- Ziel des Projekts ist ein zentrales, vollumfängliches **Betriebssystem für den MERSCH75** — digitale Souveränität, Automatisierung operativer Prozesse, zentrale Datenhaltung („Single Source of Truth").
- **Hosting** (**geplant**): dedizierte/virtuelle Server bei **Hetzner** oder **Hostinger**.
 - **Containerisierung** (**vorbereitet**): `Dockerfile` + `docker-compose` + `README-HETZNER.md` im Repo → providerunabhängiges Deployment.
 - **Übergabefähigkeit** (**Designziel**): TypeScript/React/Node.js, modular & dokumentiert, damit jedes IT-affine Komiteemitglied warten kann (kein Single Point of Failure).

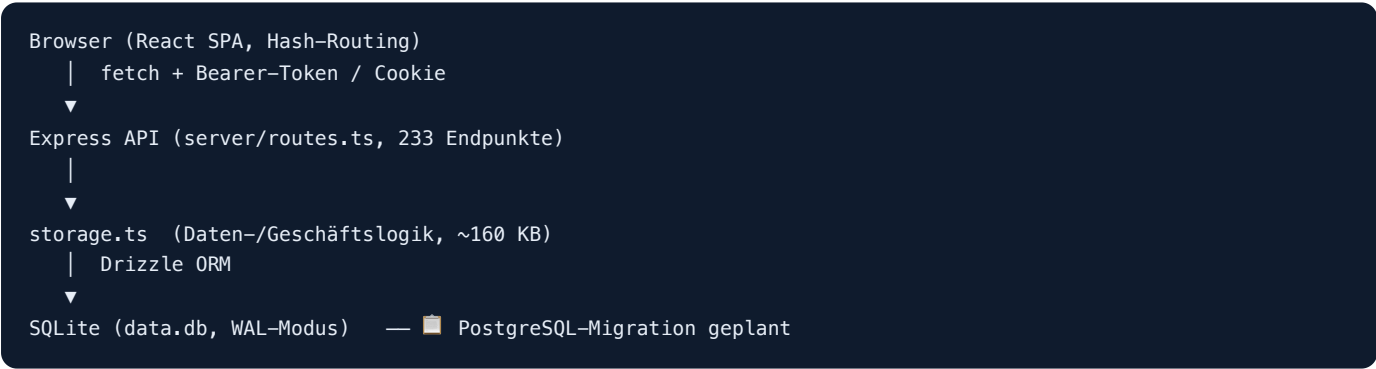
2. Kennzahlen (gemessen am Code)

Metrik	Wert	Anmerkung
API-Endpunkte (routes.ts)	233	app.get/post/put/patch/delete
Datenbank-Tabellen (schema.ts)	80	Drizzle sqliteTable (verifiziert)
Frontend-Seiten	35	client/src/pages/*.tsx
useQuery -Aufrufe	93	React-Query-Anbindung
shadcn/ui-Komponenten	47	components/ui
Backend-Code	~7.990 Zeilen	server/*.ts
Frontend-Seiten-Code	~8.713 Zeilen	nur pages/
App-Code gesamt	~16.700 Zeilen	
storage.ts / routes.ts / schema.ts	~160 / ~102 / ~73 KB	Daten-Schicht / API / Modell

Einordnung: Größenordnung eines kleinen kommerziellen SaaS.

3. Architektur & Programmaufbau

3.1 Schichten



3.2 Frontend (✅)

- **React + TypeScript + Vite**, Routing via **wouter** (useHashLocation → läuft auch in iframes).
- **State/Cache:** TanStack React Query (93 Queries).
- **UI:** TailwindCSS + **shadcn/ui** (47 Komponenten).
- **i18n:** i18next + Browser-Detector, **DE/FR/EN/LU**, Persistenz localStorage (m75_lang), Fallback DE.
- **Auth:** lib/auth.tsx — Passwort-, Karten- (Random-No) und Admin-Login; Bearer-Token im Speicher.
- **Fehler-Robustheit:** **ErrorBoundary** (29.06.) umschließt Seiteninhalt → kein Totalabsturz.

3.3 Backend (✅)

- **Node.js + Express + TypeScript**, Einstieg server/index.ts .
- **Sicherheits-Middleware** (security.ts , ~10 KB): Rate-Limiting (100 Req/15 min), Threat-Detection, Security-Header, Audit-Logging.

- **Spezial-Module:** `flhImport.ts` (handball4all.de), `import.ts` (Excel-Import), `importSboLinks.ts` (SBO-Berichte), `email.ts` (SMTP/Templates), `auth.ts` .
- **ORM:** Drizzle + better-sqlite3; `drizzle-kit push` für Schema-Sync.

3.4 Bewertung

- **Stark:** klare Schichtung, zentrale Storage-Schicht, typisiertes Schema, konsequentes UI-System, Security-Middleware.
- **Schulden:** `routes.ts` / `storage.ts` sind **Monolith-Dateien**; SQLite-Grenze (`data.db-wal` ~4 MB aktiv) → PostgreSQL nötig für Mehr-Vereins-Betrieb; **keine automatisierten Tests**; **Demo-Passwörter** im Klartext → vor Produktiv rotieren.

4. Visuelles Konzept (✅)

- **Design-Sprache:** iOS-/„Apple-style“ — abgerundete Karten (`rounded-2xl`), Blur-Header (`backdrop-blur-xl`), schwebende Sidebar mit Verlauf (`#002F65` → `#00193a`), Akzent **Vereinsgelb** `#FFDE00` .
- **Layout:** feste Desktop-Sidebar (268 px), Sektionen *Übersicht* / *Verein* / *Sport* / *Verwaltung*; mobil **Sheet-Menü** + **iOS-Tab-Bar**; sticky Header mit Theme-Toggle.
- **Dark/Light Mode** (`lib/theme`).
- **Konsistenz:** durch shadcn/ui einheitlich (Buttons, Cards, Inputs, Toasts, Tooltips).

5. Datenbank-Aufbau & Rollen (RBAC) (✅ Kern / 🟦 RBAC-Feinschliff)

- **Struktur:** relationale DB (SQLite, ausgelegt auf PostgreSQL). 80 Tabellen bündeln alle Geschäftsbereiche → Single Source of Truth.
- **RBAC:** granulare Berechtigungen — Kassenwart → Finanzen, Trainer → Trainingsplan/Anwesenheit, Vorstand → Admin, Mitglied → Profil/Rechnungen/Chat. (Rollensteuerung in `Layout.tsx` sichtbar.)

6. Modul-Status (verifiziert)

6.1 ✅ Fertig & nutzbar

Login (3 Wege) · Spiele inkl. FLH-Import · Beiträge (`Fees`) · Check-in/QR (`CheckIn`) · Trainingsplan · Statistiken/Berichte · Anwesenheit · Kalender (ICS) · Dokumente · Mitglieder-Import (Excel) · E-Mail-Einstellungen · Spielerstatistiken · Finanzen.

6.2 🟦 Funktioniert, ausbaufähig

Anmeldungen (intern) · Mitglied-Detail · Nominierungen · Ankündigungen · Chat (interne Tabelle) · Mitglieder · Öffentl. Anmeldung · Profil · Dashboard · Website-Hub (`Website.tsx` , `iframe`) · Willkomm-Mapp · Teams/Meetings.

6.3 🟡 Platzhalter-Stubs (generisches CRUD, je ~89 Z.)

`Budget` · `Duties` · `Facilities` · `Gallery` · `GdprTools` · `Newsletter` · `Shop` · `Sponsors` · `Waitlist` .

6.4 🟩 Nur Datenmodell (Tabelle ohne UI)

`referees` , `referee_assignments` · `inventory_items` , `inventory_loans` · `injuries` , `rehab_sessions` · `polls` , `poll_options` , `poll_votes` · `carpools` , `carpool_passengers` · `opponents` , `opponent_history` , `match_reports` ·

duty_rosters , duty_swaps · fan_content , live_ticker · external_calendars , calendar_sync_logs · archive_* (Saison-Archiv) · sepa_mandates , sepa_transactions · family_links , document_expiries , document_signatures , user_notifications , player_flags , gdpr_consent , facility_bookings .

7. Funktionale Module — Detail mit Reifegrad

7.1 Sport (✅) — Spiele, Teams, Training, Nominierung, Statistik, Check-in. Echte FLH-Datenanbindung über flhImport.ts .

7.2 Finanzmanagement (✅ Kern / 🧩 SEPA)

- ✅ Mitgliedsbeiträge, Zahlungen, offene Posten, Zahlungsaufforderungen (Fees , Finance , fee_rules , fee_payments , member_fees , transactions).
- 🧩 SEPA (sepa_mandates , sepa_transactions): Tabellen da, UI fehlt.

7.3 Eventmanagement (📅 / 🧩)

events , event_rsmps , generated_events vorhanden. Einladungs-/Zu-/Absage-Workflow ausbaufähig.

7.4 Bestellungs-/Materialmanagement (🟡 / 🧩)

Shop.tsx (Stub) + shop_products , shop_orders ; Material via inventory_items / inventory_loans (🧩). Voller Bestell-Workflow = Ausbau.

7.5 Kassensysteme Getränke & Eintritt (📅 Idee)

Nicht im Code — keine Tabellen. Konzept: digitale Strichliste/Getränke-Bestand + Ticketverkauf gekoppelt an qr_checkins . **Nächster Bauauftrag, falls gewünscht.**

7.6 Sponsoring & Partnerschaften (🟡 → 📅 Ausbau)

sponsors + Sponsors.tsx als Stub. Vertrags-/Leistungs-/ROI-Tracking & Zahlungsüberwachung sind **Ausbau-Ideen** (Datenmodell muss erweitert werden).

7.7 Bénévole-/Ehrenamt-Management (🧩 → 📅)

duties , duty_rosters , duty_swaps vorhanden. **Stundenerfassung, Statistik, Aufwandsentschädigung** noch nicht modelliert → Konzept.

7.8 Erweitertes Finanzwesen: Gehälter, Subventionen, Fördermittel (📅 Idee)

Nicht im Code. Payroll, Einnahmen-Klassifizierung (Staat/Gemeinde/Beiträge/Spenden), Fördermittel-DMS, Guichet.lu-Linkdatenbank → komplettes neues Modul.

7.9 Compliance, Lizenzen & RGPD (🧩 / 🟡)

- 🧩 Lizenz-/INAPS-Tracking-Basis: player_flags , document_expiries . **Automatisches Nominierungs-Blocking** = Ausbau-Idee.
- 🟡 RGPD: gdpr_consent + GdprTools.tsx (Stub). Foto-Einwilligungen modelliert, Auto-Filter = Ausbau.

7.10 Infrastruktur & Fahrgemeinschaften (🟡 / 🧩)

- 🟡 Hallenbelegung: Facilities.tsx (Stub) + facilities / facility_bookings . Doppelbuchungs-Sperre = Ausbau.

- 🍀 Carpools: `carpools` / `carpool_passengers` (Tabelle, keine UI). Auto-Zuteilung/Routen = Idee.

7.11 Website-Integration (📘 Hub / 📄 API)

- 📘 `Website.tsx` zeigt öffentliche Seiten per iframe-Vorschau + Edit-/Repo-Links; `website_pages` -Tabelle vorhanden.
- 📄 **Headless-Read-API**, die die statische Website automatisch mit Live-Daten (Tabelle, nächste Spiele, Ankündigungen) speist, ist **geplant**, noch nicht gebaut. **Hinweis:** Website ist **statisch (GitHub Pages)**, nicht WordPress.

8. Geplante Meta-Schicht: Hermes & Odysseus (📄 Konzept)

Status: noch kein Code. Sinnvolle, klar umrissene Roadmap.

- **Hermes (Monitoring & Orchestrierung):** überwacht Host-Server (Hetzner/Hostinger), DB-Integrität, Container-Status; alarmiert das Komitee (z. B. via Matrix).
- **Odysseus (Daten-Aggregation):** Playwright-basierter Agent für Web-Scraping/API-Abfragen bei Drittsystemen (FLH), um externe Daten automatisch zu injizieren. **Heute** erledigt das der Server-Import `flhImport.ts` (ohne Browser-Automation).

9. Disaster Recovery & Datensicherheit (📄 Konzept)

- **Geplant:** n8n-Workflows erzeugen nächtliche DB-Dumps (SQLite/PostgreSQL) + Nextcloud-Snapshots, **verschlüsselt** auf lokales **QNAP-NAS (192.168.178.5)** im Vereinsnetz gespiegelt → Wiederherstellung bei Cloud-Ausfall.
- **Heute:** kein Backup-Code im Repo. **Empfehlung: priorisieren**, sobald Produktivbetrieb startet.

10. Komplexität & Risiken

Dimension	Bewertung	Begründung
Funktionsumfang	Hoch	35 Seiten, 80 Tabellen, 233 Endpunkte
Code-Struktur	Mittel-Hoch	gute Schichtung, aber Monolith-Dateien
Wartbarkeit	Mittel	große Dateien, keine Tests
Skalierbarkeit	Mittel	SQLite-Limit; PostgreSQL geplant
Sicherheit	Mittel-Hoch	Middleware da; Demo-PW/Secrets offen
Reifegrad UI	Gemischt	Top-Seiten vs. 9 Stubs + viele 🍀 -Tabellen
Doku	Sehr hoch	PROJEKTPLAN + Feature-Doku

11. Externe Tools, KI & Systeme (📅 alle geplant)

Leitlinie: Open Source first, selbst-gehostet, kostenlos, RGPD-konform; KI-Daten verlassen den Server möglichst nicht.

Tool	Rolle	Status
Ollama	lokale LLM-Laufzeit (Chat, RAG, Textgenerierung, Spielberichte, Wöllkomm-Mapp)	📅
Matrix (Synapse)	selbst-gehostetes, E2E-verschlüsseltes Chat-Protokoll	📅
Element	Matrix-Client (Web/Mobile) für Mitglieder	📅
n8n	visuelle Workflow-Automatisierung (FLH→DB→Matrix/E-Mail; Neumitglied→Mappe→Rechnung)	📅
Uptime Kuma	Monitoring (HTTP, Zertifikate, Heartbeats, Webhook-Alarm)	📅
Mautrix-Bridges	Brücke Matrix ↔ WhatsApp/Signal	📅
Nextcloud	Datenspeicher für Großdateien + CalDAV	📅

11.1 Empfohlene KI-Architektur mit Update-Möglichkeit

Adapter-Muster + Admin-UI, damit Modelle/Anbieter ohne Code-Änderung wechselbar sind:

```
ai_providers (type: ollama|openai|anthropic|localai|custom, base_url, model, embed_model, api_key_enc, enabled, i
ai_jobs      (Audit: Prompt, Modell, Tokens, Dauer, Nutzer)
ai_documents (RAG-Quellen: Datei, Chunks, Embeddings)
```

- OpenAI-kompatible `aiService` -Abstraktion (Ollama/LocalAI/OpenAI/Anthropic gleiches Format).
- Admin „KI-Einstellungen“: Anbieter/Modell wählen & testen, `GET /api/ai/models` , `POST /api/ai/pull` (Modell laden/aktualisieren), Keys **verschlüsselt**.
- RAG: PDFs → Chunking → Embeddings (`nomic-embed-text`) → Vektor-Store (`sqlite-vec` /Chroma) → Retrieval → LLM → Antwort **mit Quellen**.

12. Nächste sinnvolle Schritte (priorisiert)

1. **Stubs ehrlich kennzeichnen** + 2–3 auf vorhandene Tabellen real ausbauen (Sponsoren, Galerie, Saison-Archiv).
2. `GET /api/health` + **Uptime-Kuma**-Monitoring (Basis für „Hermes“).
3. **Öffentliche Read-API** + 1 Website-Widget (Ligatabelle) als sichtbarer Quick-Win.
4. **KI-Fundament** (Ollama + `ai_providers` + Admin-UI mit Pull/Update).
5. **Secrets bereinigen** (Demo-PW raus), erste **Vitest/Playwright**-Tests.
6. **Backup-Konzept** (QNAP-Spiegelung) umsetzen, sobald Produktivbetrieb startet.

Hinweis: Reifegrad-Angaben basieren auf statischer Code-Analyse vom 29.06.2026 (Tabellen via `grep` aus `schema.ts` , Seiten aus `client/src/pages/`). Mit 📅 markierte Bausteine sind Vision/Roadmap ohne Code.